

Indexation : Fondamentale et Avancée



Samuel Rohaut

Qu'est-ce qu'un index ?

Structure de données auxiliaire qui accélère l'accès aux données

Analogie avec l'index d'un livre

Compromis entre vitesse de lecture et coût d'écriture

Occupe de l'espace disque supplémentaire

Fonctionnement général

Stocke des valeurs de clés avec des pointeurs vers les lignes correspondantes

Organise ces valeurs de manière à permettre des recherches efficaces

Réduit considérablement les I/O disque lors des requêtes

Quand utiliser un index ?

Tables volumineuses (>1000 lignes)

Colonnes fréquemment utilisées dans les clauses WHERE, JOIN, ORDER BY

Colonnes avec forte sélectivité (nombreuses valeurs distinctes)

Contraintes d'unicité (PRIMARY KEY, UNIQUE)

Les index fondamentaux

Types d'index fondamentaux

B-tree (par défaut)

Structure de données arborescente équilibrée

Utilisations:

Égalité (=)

Intervalles (<, >, BETWEEN)

Préfixes (LIKE 'abc%')

Tri (ORDER BY)

Le plus polyvalent, choix par défaut

Supporté pour pratiquement tous les types de données

Exemple

```
CREATE INDEX idx_customers_last_name ON customers(last_name);
```

Cas d'utilisation: Base de données clients où vous recherchez fréquemment par nom (égalité) ou réalisez des tris alphabétiques.

Parfait pour des requêtes comme :

```
SELECT * FROM customers WHERE last_name = 'Smith'
```

```
SELECT * FROM customers ORDER BY last_name.
```

Types d'index fondamentaux

Hash

Basé sur des fonctions de hachage

Utilisations:

- Uniquement pour les recherches d'égalité (=)

- Pas pour les intervalles ni le tri

Potentiellement plus rapide pour les égalités simples

Limitations:

- Pas transactionnel avant PostgreSQL 10

- Ne supporte pas les index multicolonne efficacement

Exemple

```
CREATE INDEX idx_products_sku_hash ON products USING HASH(sku);
```

Cas d'utilisation: Système d'inventaire avec recherche exacte par code produit. Efficace pour `SELECT * FROM products WHERE sku = 'ABC123'`, mais inutile pour les recherches par plage ou les tris.

Types d'index fondamentaux

GiST (Generalized Search Tree)

Structure d'index extensible pour données complexes

Utilisations:

- Données géospatiales

- Texte avec recherche de similarité

- Valeurs d'intervalle

Moins efficace que B-tree pour les cas simples

Essentiel pour les données non linéairement ordonnables

Exemple

```
CREATE INDEX idx_cities_location ON cities USING GIST(location);
```

Cas d'utilisation: Application cartographique nécessitant de trouver toutes les villes dans un rayon de 50km autour d'un point.

Permet des requêtes comme :

```
SELECT * FROM cities  
WHERE ST_DWithin(location, ST_MakePoint(long, lat), 50000)
```

Types d'index fondamentaux

GIN (Generalized Inverted Index)

Index inversé pour valeurs composites

Utilisations:

- Recherche full-text

- Arrays

- JSONB

Très performant pour la recherche dans des structures composites

Construction plus lente, espace disque plus important

Exemple

```
CREATE INDEX idx_articles_tags ON articles USING GIN(tags);
```

Cas d'utilisation: Plateforme de blog où chaque article possède plusieurs tags (stockés dans un tableau).

Optimise les requêtes comme :

```
SELECT * FROM articles WHERE tags @> ARRAY['postgresql', 'database']
```

Types d'index fondamentaux

BRIN (Block Range INdex)

Index minimaliste stockant des métadonnées par bloc

Utilisations:

- Très grandes tables (téraoctets)

- Données physiquement ordonnées (timestamps, IDs séquentiels)

- Extrêmement compact (99% plus petit que B-tree)

- Performance moindre mais rapport taille/bénéfice excellent

Exemple

```
CREATE INDEX idx_logs_timestamp ON logs USING BRIN(created_at);
```

Cas d'utilisation: Table de logs historiques de plusieurs téraoctets avec des entrées chronologiques.

Très efficace pour des requêtes comme `SELECT * FROM logs WHERE created_at BETWEEN '2023-01-01' AND '2023-01-31'` avec un impact minimal sur le stockage.

Les index avancés

Techniques d'indexation avancées

Index multicolannes (composites)

Combine plusieurs colonnes dans un seul index

L'ordre des colonnes est crucial:

- Premières colonnes utilisables seules

- Colonnes suivantes utilisables uniquement si les précédentes ont une condition d'égalité

Utilisations:

- Filtres sur plusieurs colonnes simultanément

- Optimisation de requêtes avec WHERE et ORDER BY

Exemple

```
CREATE INDEX idx_orders_customer_date ON orders(customer_id, order_date);
```

Cas d'utilisation: Système de gestion de commandes où vous recherchez fréquemment les commandes d'un client dans une période spécifique.

Optimise :

```
SELECT * FROM orders WHERE customer_id = 123 AND order_date > '2023-01-01'
```

Techniques d'indexation avancées

Index partiels

Indexe seulement un sous-ensemble des lignes

Défini avec une clause WHERE

Avantages:

- Plus petit (moins d'espace disque)

- Maintenance moins coûteuse

- Potentiellement plus rapide

Exemples:

- Index sur les produits actifs uniquement

- Index sur les transactions récentes

Exemple

```
CREATE INDEX idx_invoices_unpaid ON invoices(due_date) WHERE status = 'unpaid';
```

Cas d'utilisation: Système de gestion de commandes où vous recherchez fréquemment les commandes d'un client dans une période spécifique.

Optimise :

```
SELECT * FROM orders WHERE customer_id = 123 AND order_date > '2023-01-01'
```

Techniques d'indexation avancées

Index d'expressions

Indexe le résultat d'une expression plutôt qu'une colonne

Utilisations:

Recherches insensibles à la casse (`UPPER(name)`)

Fonctions sur dates (`EXTRACT(YEAR FROM date)`)

Transformations courantes (calculer un ratio)

La requête doit utiliser exactement la même expression

Exemple

```
CREATE INDEX idx_users_email_lower ON users(LOWER(email));
```

Cas d'utilisation: Système d'authentification avec recherches d'email insensibles à la casse.

Optimise les requêtes comme

```
SELECT * FROM users WHERE LOWER(email) = 'user@example.com'
```

Techniques d'indexation avancées

Index couvrants (INCLUDE)

Stocke des données supplémentaires non indexées

Avantage: évite l'accès à la table (index-only scan)

Différence avec index composite:

- Colonnes INCLUDE non utilisables pour le filtrage

- Plus efficace en espace pour les larges colonnes

Exemple

```
CREATE INDEX idx_products_category_include_name_price  
ON products(category_id) INCLUDE (name, price);
```

Cas d'utilisation: Catalogue de produits où vous affichez fréquemment les noms et prix par catégorie. L'index contient toutes les données nécessaires sans accéder à la table :

```
SELECT name, price FROM products WHERE category_id = 5
```


Indexation pour cas spécifiques

Indexation pour cas spécifiques

Recherche textuelle

Options:

- B-tree avec LIKE (limité aux préfixes)

- Trigram indexes (GIN ou GiST, recherche de sous-chaînes)

- Full-text search (tsvector/tsquery avec GIN)

Trade-offs:

- Précision vs performance

- Espace disque

- Complexité d'implémentation

Exemple

```
CREATE INDEX idx_documents_fulltext  
ON documents USING GIN(to_tsvector('french', content));
```

Cas d'utilisation: Moteur de recherche documentaire permettant de trouver des documents contenant certains termes.

Optimise les requêtes comme :

```
SELECT title FROM documents  
WHERE to_tsvector('french', content) @@  
      to_tsquery('french', 'indexation & performance')
```

Indexation pour cas spécifiques

Données JSON/JSONB

JSONB supporte plusieurs types d'index:

- GIN pour recherche de clés/valeurs

- B-tree pour chemins spécifiques via ->>

Opérateurs spécialisés:

- ? (existence de clé)

- @> (contenance)

- jsonb_path_ops pour optimiser @>

Exemple

```
CREATE INDEX idx_profiles_skills ON profiles USING GIN(data->'skills');
```

Cas d'utilisation: Plateforme de recrutement avec profils de compétences en JSON.

Permet des requêtes efficaces pour trouver des candidats avec compétence spécifique :

```
SELECT * FROM profiles WHERE data->'skills' ? 'PostgreSQL'
```

Indexation pour cas spécifiques

Données géospatiales

Utilisation principale: GiST

Extensions:

- PostGIS pour géospatialité avancée

- Cube pour espaces multidimensionnels

Opérateurs de distance, contenance, intersection

Exemple

```
CREATE INDEX idx_restaurants_location  
ON restaurants USING GIST(ST_SetSRID(location, 4326));
```

Cas d'utilisation: Application de livraison de repas nécessitant de trouver tous les restaurants dans un quartier spécifique.

Permet des requêtes comme :

```
SELECT * FROM restaurants  
WHERE ST_Contains(ST_MakePolygon($polygon), location)
```

Analyse et maintenance

Analyse et maintenance

Analyse de l'utilisation des index

Commande EXPLAIN (ANALYZE)

Interpréter:

Index Scan vs. Sequential Scan

Bitmap Index Scan

Index Only Scan (optimal)

Statistiques: pg_stat_user_indexes

Analyse et maintenance

Maintenance des index

VACUUM pour récupérer l'espace

REINDEX pour reconstruire un index fragmenté

ANALYZE pour mettre à jour les statistiques

Surveillance de la fragmentation

Considérations de performance:

- Moments opportuns (heures creuses)

- Impact sur les performances

Analyse et maintenance

Bonnes pratiques

Ne pas surindexer:

- Chaque index ralentit les écritures

- Consomme de l'espace disque

- Ralentit VACUUM

Supprimer les index inutilisés

Privilégier les index composites aux index multiples

Tester les performances réelles

En résumé

Les index sont essentiels pour les performances

Le choix du type d'index dépend:

- Du type de données

- Des opérations fréquentes

- Des patrons d'accès

L'indexation avancée peut transformer des requêtes lentes en requêtes rapides

La surveillance et maintenance des index sont aussi importantes que leur création

L'optimisation est un processus continu, pas une action ponctuelle

Mise en pratique avec le TP #03